

## **PROGRAM**

```
# Data Ingestion and Preprocessing
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

print("Tebin Sebastian-64")

california = fetch_california_housing(as_frame=True)
df = california.frame
X = df['AveRooms'].values
y = df['MedHouseVal'].values.reshape(-1, 1)
print("Dataset Loaded successfully.")
print(f"Feature Matrix Shape (X): {X.shape}")
print(f"Target Vector Shape (y): {y.shape}\n")
scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)
m = len(y)
df.describe()
df.head()
X_b = np.c_[np.ones((m, 1)), X_scaled]

# Model Implementation
theta_normal_scaled =
np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y_scaled)
y_pred_normal_scaled = X_b.dot(theta_normal_scaled)
y_pred_normal = scaler_y.inverse_transform(y_pred_normal_scaled)
def compute_cost(X, y, theta):
    m = len(y)
    predictions = X.dot(theta)
    cost = (1 / (2 * m)) * np.sum((predictions - y) ** 2)
    return cost
def gradient_descent(X, y, theta, learning_rate, iterations):
    m = len(y)
    cost_history = []
    for i in range(iterations):
        gradients = (1 / m) * X.T.dot(X.dot(theta) - y)
        theta = theta - learning_rate * gradients
        cost_history.append(compute_cost(X, y, theta))
    return theta, cost_history
learning_rate = 0.01
iterations = 1500
initial_theta = np.zeros((2, 1))
theta_gd_scaled, cost_history = gradient_descent(
    X_b,
    y_scaled,
    initial_theta,
    learning_rate,
    iterations
)
y_pred_gd_scaled = X_b.dot(theta_gd_scaled)
y_pred_gd = scaler_y.inverse_transform(y_pred_gd_scaled)

# Performance Metric Evaluation
mse_normal = mean_squared_error(y, y_pred_normal)
r2_normal = r2_score(y, y_pred_normal)
mse_gd = mean_squared_error(y, y_pred_gd)
r2_gd = r2_score(y, y_pred_gd)
print("MODEL EVALUATION RESULTS ")
print(
    f"Normal Equation:\n"
    f"- Mean Squared Error (MSE): {mse_normal:.4f}\n"
    f"- R-squared (R2): {r2_normal:.4f}"
)
print(
    f"Gradient Descent:\n"
    f"- Mean Squared Error (MSE): {mse_gd:.4f}\n"
    f"- R-squared (R2): {r2_gd:.4f}\n"
)
```

```
print(
    f"Final Weights (Scaled) - Normal Equation: "
    f"Intercept = {theta_normal_scaled[0][0]:.4f}, "
    f"Slope = {theta_normal_scaled[1][0]:.4f}"
)
print(
    f"Final Weights (Scaled) - Gradient Descent: "
    f"Intercept = {theta_gd_scaled[0][0]:.4f}, "
    f"Slope = {theta_gd_scaled[1][0]:.4f}"
)

# Visual Target Data Curves
sort_axis = np.argsort(X.flatten())
X_sorted = X[sort_axis]
y_pred_gd_sorted = y_pred_gd[sort_axis]
plt.figure(figsize=(10, 6))
plt.scatter(
    X[:1000],
    y[:1000],
    color="gray",
    alpha=0.5,
    label="Actual Data Points (Subset)"
)
plt.plot(
    X_sorted,
    y_pred_gd_sorted,
    color="crimson",
    linewidth=3,
    label="Fitted Regression Line"
)
plt.title(
    "Linear Regression: AveRooms vs Median House Value",
    fontsize=14
)
plt.xlabel(
    "Average Rooms Per Dwelling (AveRooms)",
    fontsize=12
)
plt.ylabel(
    "Median House Value (in $100,000s)",
    fontsize=12
)
plt.xlim(0, 15)
plt.ylim(0, 5.5)
plt.legend(fontsize=11)
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()

# Plot Convergence Curve History
plt.figure(figsize=(8, 4))
plt.plot(
    range(iterations),
    cost_history,
    color="navy",
    linewidth=2
)
plt.title("Gradient Descent Cost Convergence Curve")
plt.xlabel("Iterations")
plt.ylabel("Cost J(theta)")
plt.grid(True, linestyle=":")
plt.show()
```